# DISTRIBUTED SYSTEMS

Lecture 2: Part 1: Intro to Distributed Systems

**What are they and why do we need them?**

# Lecture Outline



*What we will cover today*

### Part 1: Intro to Distributed Systems

- **What is a DS?**
- **Why DS?**
- **Real-world Examples (Facebook,Google, Amazon)**

**Activity & Lab Demo**

Discussion: Why use multiple computers instead of one supercomputer?

# What is a Distributed System?



*Definition (Coulouris et al., p.2)*

- *A distributed system is a collection of independent computers that appears to users as a single coherent system. (Coulouris)*
- Distributed computing studies how to design systems in which independent computers, connected by a network, coordinate their actions and share resources by passing messages — so that together they behave like a single system.

**Main Characteristics:**

- **Concurrency:** Components execute independently at the same time.
- **No Global Clock:** Components cannot rely on perfect time synchronization.
- **Independent Failures:** Components can fail independently without bringing down the whole system.
- **Message Passing: The ONLY way components interact through network messages.**

**The meaning of:**

**Independent Computers**

Each computer: Has its own memory, Has its own processor, Can fail independently

**Single System View**

Users see only one service.

# Key Aspects of This Definition



*Definition*

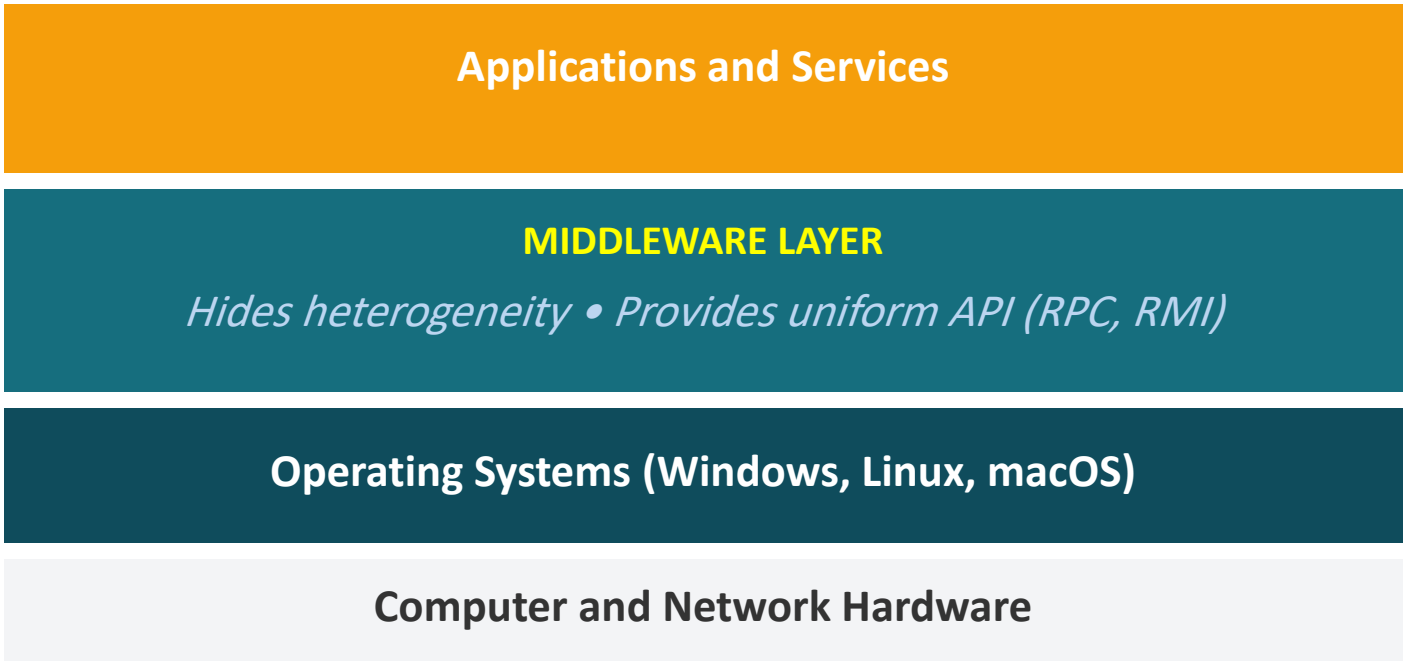
1. Autonomous Components: Independent computers operating on their own. Each has its own processor, memory, and OS.
2. Single System View: Although many computers are working together, users think they are using one system.
3. Collaboration Required: The computers cannot work alone. They must cooperate and exchange information.
4. No Assumptions About Computer Types or Interconnections: Distributed systems can contain different types of computers and networks (Windows, Linux, Wi-Fi, Fiber).

**Key Idea: Computers are separate and independent, but they must coordinate and communicate to appear as one.**

# The Role of Middleware



*The main layer to make the system appears to users as a single coherent system.*




*Middleware makes a distributed system appear as a single coherent system to the application developer.*

# Distributed System as Middleware

*Organization*

**Middleware is software that sits between applications and the operating system/network.**

- Its job is to help different computers communicate and work together.

Example: A banking app on a phones communicates with bank servers through middleware.

*Analogy: Middleware is like a translator helping people who speak different languages.*

# Main Points of Middleware



*Organization*

1. Logically Placed Between Apps and OS: Applications do not directly interact with hardware or operating systems.
2. Distributed Apps: One application can run on multiple computers (e.g., logic on one, database on another).
3. Same Interface Everywhere: Applications use services by the same way regardless of where resources are located.
4. Hides Heterogeneity: Applications do not need to know which processor, OS, or computer stores the data.

**Benefits: Hides differences, provides a common interface, supports distribution, and simplifies development.**

- There are different types of system architecture

  Centralized - Decentralized - Distributed systems
- How computers are organized

  Centralized: One master server

  Decentralized: Few regional masters (more than one master server)

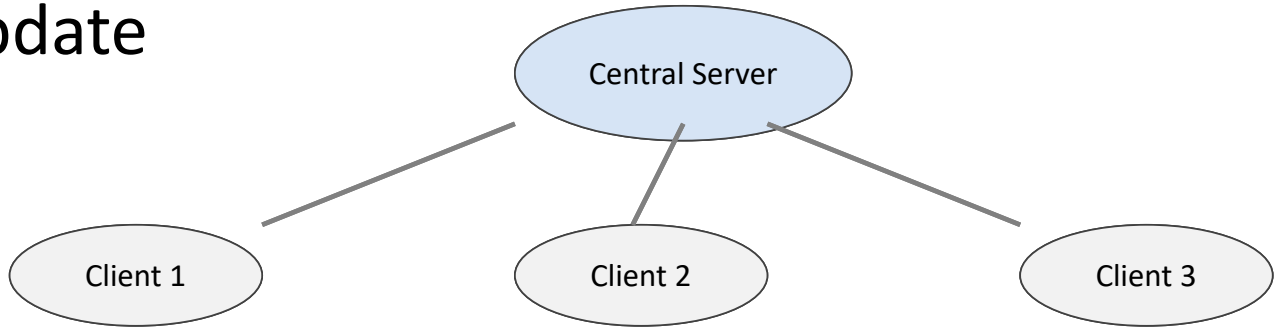
  Distributed:

  - All peers are equal
  - No single point of failure

# Centralized System



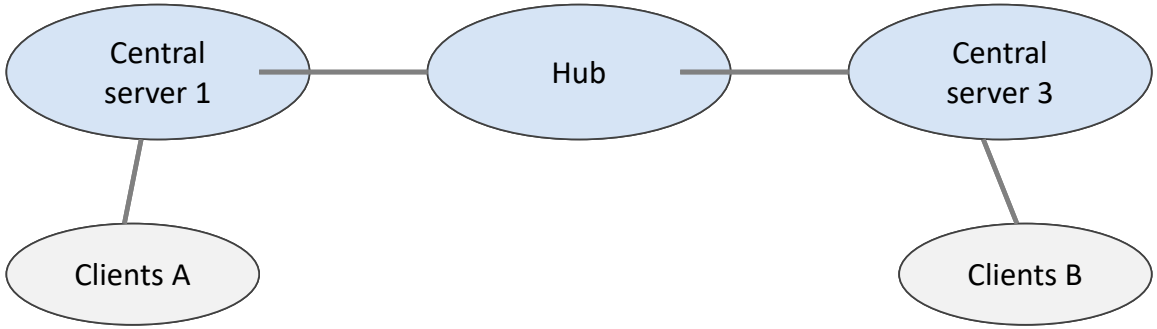
- One central server controls all clients
- Clients depend completely on server
- Easy to manage and update
- Simple architecture
- Single point of failure
- Server overload is common



# Decentralized System



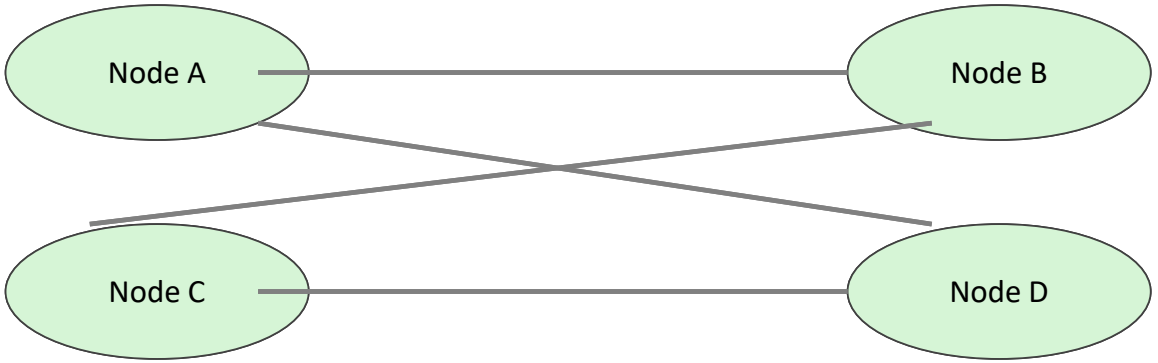
- Multiple regional central nodes
- Power shared among hubs (central servers communicated with each other)
- No single master controls all nodes
- Better fault tolerance (not depend only on one central server)
- Harder to maintain globally



# Distributed System



- All nodes are equal peers.
- No central master node exists.
- Nodes coordinate via messages.
- Highly fault tolerant.
- Very high scalability.
- Complex to design securely.



# Quick Comparison



| Feature | Centralized System | Decentralized System | Distributed System |
|---|---|---|---|
| Control Structure | One Master | Few Masters | All Peers |
| Management | Simple to manage | Balanced management | Shared management |
| Fault Tolerance | Low | Medium | High |
| Failure Impact | One crash stops the entire system | Partial crash survival | Survives node crashes |
| Scalability | Limited | Better than centralized | Highly scalable |
| Examples | Single Server Database | Multiple Regional Servers | Peer-to-Peer Network |
| **Real world examples** | • Traditional Bank Server<br>• Old ATM networks | • The DNS System<br>• Global Email servers | • Bitcoin<br>• Google Cloud, BitTorrent |

# Why Distributed Systems are needed?



**Distributed systems are designed to meet the following requirements:**

1: Resource Sharing
2: Scalability
3: Fault Tolerance

# Why Distributed Systems?



*Motivation 1: Resource Sharing*

**Types of Resources Shared**

- Hardware: Printers, storage devices, specialized processors.
- Software: Applications, services, libraries.
- Data: Files, databases, web pages, multimedia streams.

**Why Share? (or the benefit of sharing)**

- Economics: Share expensive hardware (e.g., supercomputers).
- Collaboration: Enable groups to work together (e.g., Google Docs).
- Data Access: Access remote information (e.g., Web, databases).

# Why Distributed Systems?



*Motivation 2,3: Scalability & Fault Tolerance*

## Scalability

- Handle increasing loads by adding more machines (horizontal scaling).
- No single bottleneck limit (Don't rely on one server, unlike a single powerful server).

## Fault Tolerance

- If one computer crashes, others take over.
- No single point of failure (no One component can stop everything).
- High availability: System remains operational.

# Real-World Examples



*Distributed systems in practice*

| Example | Type of Distributed System applications | Scale |
|---|---|---|
| Google Search | Web search & indexing | Indexes billions of web pages across millions of servers. |
| Facebook | Social networking | Handles millions of concurrent users and massive data streams. |
| Amazon | E-commerce & Cloud (AWS) | Processes millions of transactions and hosts global cloud services. |
| Cloud Computing | Infrastructure as a Service (IaaS) | Provides virtualized computing resources over the Internet. |

**NOTE:**
**AWS:(Amazon Web Services)** It is the **cloud computing platform provided by Amazon**.

# Example: Facebook

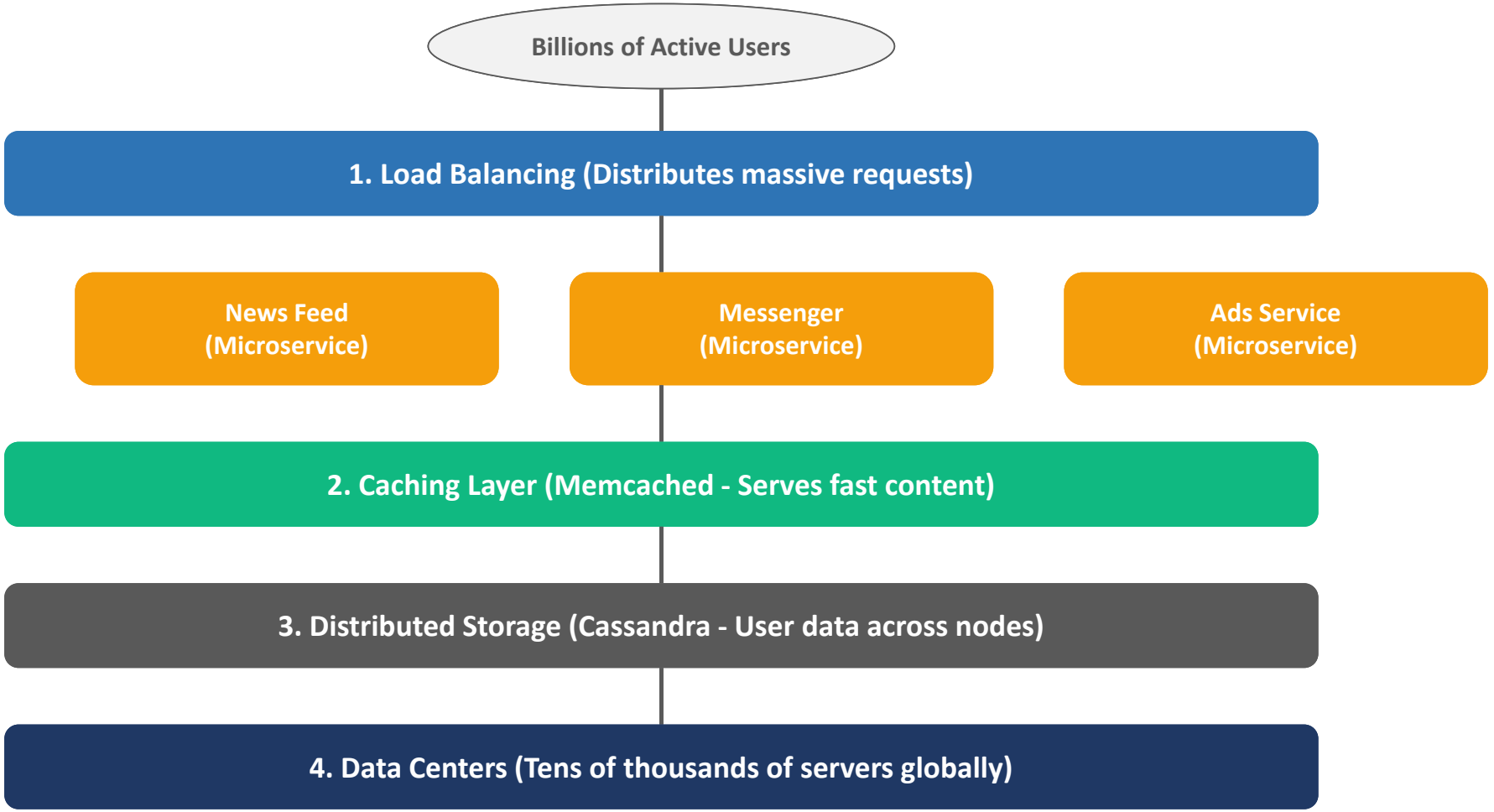


*Social networking at scale*

## Facebook's Distributed Architecture:

1. Load Balancing: Distributes billions of incoming requests across server clusters.
2. Microservices: Different features (messenger, ads, graph) run on separate distributed services.
3. Caching: Uses distributed caches to **Keep a copy nearby for quick access**.
4. Distributed Storage: Uses systems like Cassandra to distribute data across multiple nodes ("Where is the user's data stored?").
5. Data Centers: Refers to the **physical location/building** that contains thousands of servers("Where are the servers located?").

# Example: Facebook Social Networking at Scale



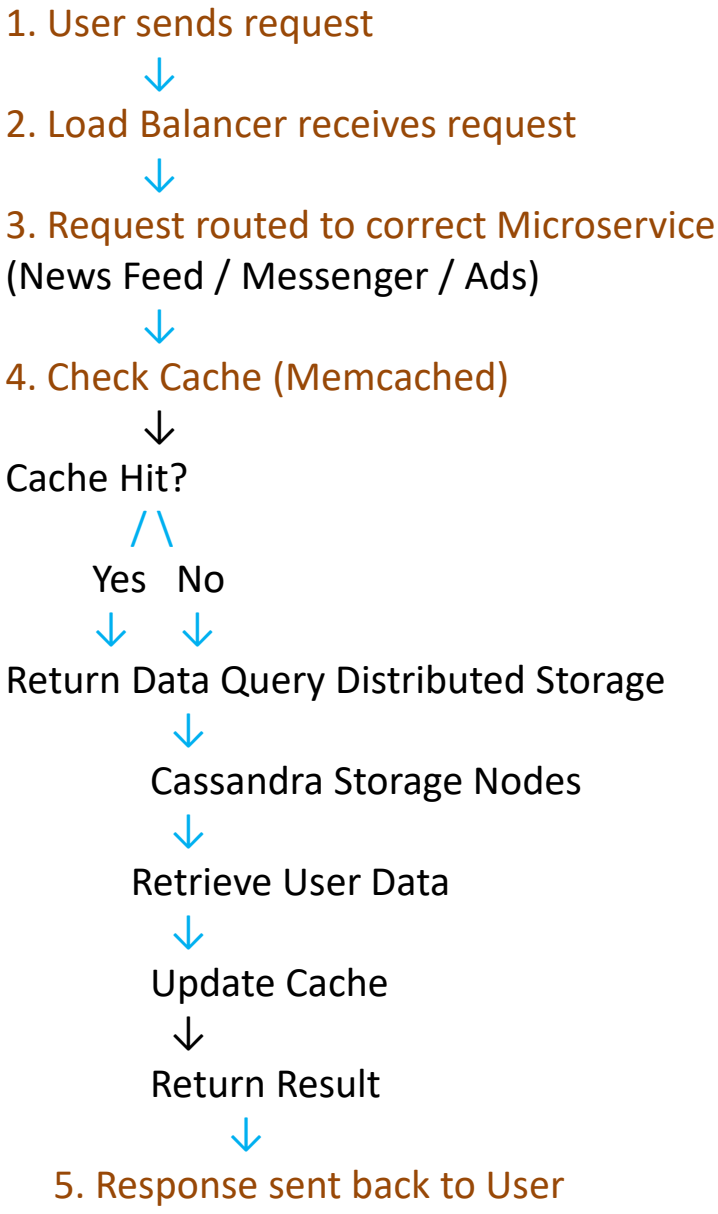
# Example: Facebook

*Social networking at scale*



## Facebook's Distributed Architecture:

- A Facebook request first goes to a **Load Balancer**,
- then to the appropriate **Microservice**,
- which checks the **Cache**; if the data is not found,
- it retrieves it from **Distributed Storage** located in Facebook's **Data Centers**,
- and finally returns the result to the user.



# DISTRIBUTED SYSTEMS

## Lecture 2:part2: Basic Concepts of Distributed Computing Systems

Definition · Components (Nodes, Processes, Links, Resource Sharing)

*Building on Lectures 1*

# Lecture Outline



*What we will cover today*

## 2. Components of Distributed Systems

- Nodes
- Processes
- Communication links
- Resource sharing

**Learning Objective**

Students will understand what distributed systems are, their core components

## Components of a Distributed System

1. Nodes
2. Processes
3. Communication Links
4. Resource Sharing

# Components: Nodes

*Component 1 of 4*



**Definition: A node is any independent, addressable computer or device that participates in a distributed system.**

**it has its own processor, memory, and network address, and can operate even if other nodes are unreachable.**

## Types of Nodes:

- High-Performance Mainframes (Supercomputers, Enterprise Servers).
- Workstations & PCs (Desktops, Laptops).
- Mobile Devices (Smartphones, Tablets).
- Sensor Nodes (IoT, Environmental monitors).
- Embedded Devices (Smart applications, Car systems).

**Roles: Client (requests), Server (provides), Peer (both). Fail independently.**

# Components: Processes

*Component 2 of 4*

**Definition: A process is a program in execution on a node. It has its own memory space and executes independently.**

## Types of Processes:

- Client Process: Requests services from servers (e.g., Web browser).
- Server Process: Provides services to clients (e.g., Web server).
- Peer Process: Acts as both client and server (e.g., BitTorrent).

**Threads: Multi-threaded servers can handle multiple clients simultaneously( Processes can migrate between nodes for load balancing).**

# Components: Communication Links



*Component 3 of 4*

**Definition: The logical channel through which nodes exchange messages, using protocols like TCP or UDP.**

Types of Networks: LAN (Ethernet), WAN (Internet), WLAN (Wi-Fi), PAN (Bluetooth).

## Communication Characteristics:

- **Latency**: Delay between send and receive.
- **Bandwidth:** Data transmitted per unit time.
- **Jitter**: Variation in delivery time.
- **Reliability**: Correct and ordered delivery.

**Models: Synchronous (wait for reply), Asynchronous (continue immediately).**

# Components: Resource Sharing

*Component 4 of 4*

**Definition: The ability of multiple processes to access and use a common resource in a controlled and consistent way.**

## Resource Sharing Mechanisms:

1. Client-Server Model: Clients request, servers respond (e.g., Web browsing).
2. Object-Based Model: Resources encapsulated as objects, accessed via methods (e.g., CORBA, Java RMI).
3. Peer-to-Peer (P2P) Model: All participants both provide and consume resources (e.g., BitTorrent).

*Example: The Web Servers manage pages, browsers request them, URLs identify them, GET/POST work everywhere.*

# Activity: Class Discussion



*Let's think about it!*

***Discussion Topic:***
***"Why do modern applications use multiple computers instead of one powerful computer?"***

## Think about these points:

1. Cost: Is it cheaper to buy one ($10M) supercomputer or (1,000) $10,000 standard PCs?
2. Scalability: What happens if your website suddenly gets 1 million visitors in one day?
3. Fault Tolerance: What happens if a single supercomputer catches fire?
4. Geography: If your users are in Tokyo, London, and New York, where do you put the server?

# Single vs Distributed system



*Comparing architectures*

### 1. Single Computer Application

- Code runs on one machine.
- Data stored in local memory/disk.
- If the machine crashes, the app dies.
- Cannot handle more load than the machine's CPU allows.

### 2. Distributed Application

- Code runs across multiple machines.
- Data partitioned/replicated.
- If one machine crashes, others take over.
- Scales horizontally by adding more machines.

# Visualizing the Difference

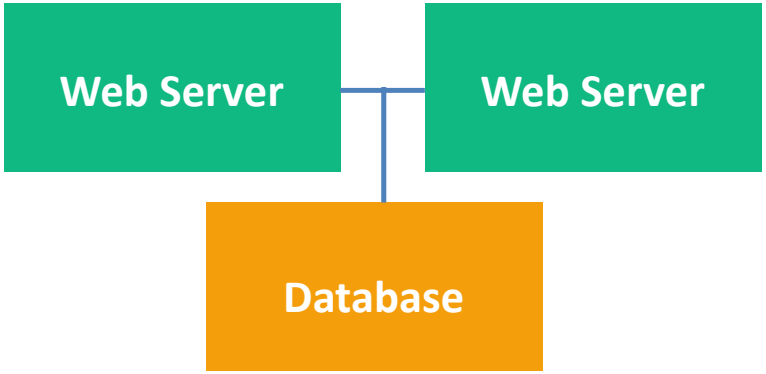


*Monolith vs Distributed*

**Single Computer App:**




**Distributed App:**




*In the lab, we can write a simple Java program (like a bank server) and show how a single server fails under load, but a distributed system can scale to handle it.*

# Summary



1. A distributed system consists of independent computers communicating via messages.
2. They are motivated by the need to share resources, scale, and tolerate failures.
3. Middleware is the key software layer that hides the complexity of the network.
4. Real-world giants like Google, Facebook, and Amazon are massive distributed systems.
5. Cloud computing is the modern utility model for delivering distributed resources.

# Summary

*Review*



## Definition & Middleware

- Collection of independent computers appearing as one.
- Middleware hides complexity and heterogeneity.
- Provides a common interface for all apps.

## Components

- Nodes: Servers, PCs, mobiles, sensors.
- Processes: Client, Server, Peer.
- Links: LAN, WAN, WLAN, PAN.
- Resource Sharing: Client-Server model, Object model, P2P model.

# LECTURE 2 - PART A

Distributed IPC, Sockets, Naming, Concurrency, and Openness (Progs 1-4)

# LECTURE 2 - PART A | PROGRAM #1 — ARCHITECTURAL FLOW

**TCP Sockets enable direct node-to-node message passing**

## SIMPLE DESCRIPTION

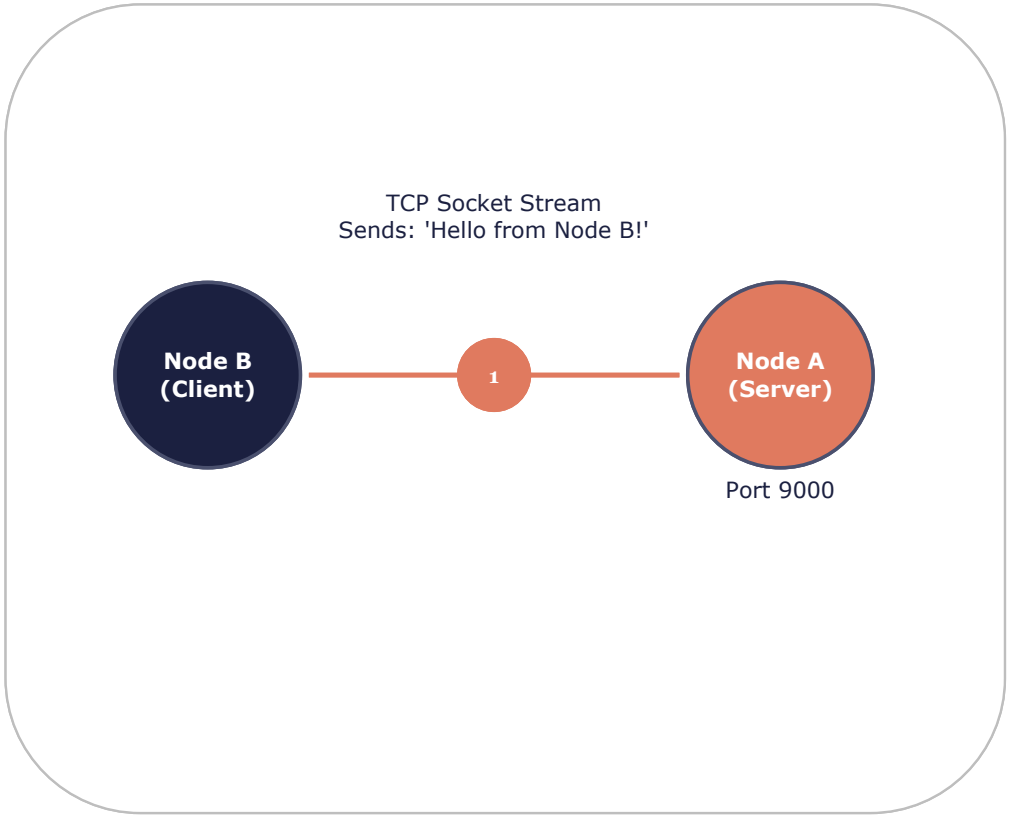
Here is a **simple program** that demonstrates **Interprocess Communication (IPC) using Message Passing over TCP Sockets**. It consists of two independent programs:

**Node A (Server)**: Waits for incoming connections and receives a message.

**Node B (Client)**: Connects to the server and sends a message.

## FUNCTION OF THE PROGRAM

This program demonstrates **Interprocess Communication (IPC)** in a distributed system. Two separate processes running independently communicate through **TCP sockets**. The client (Node B) sends a text message to the server (Node A), and the server receives and displays the message, illustrating the concept of **message passing between distributed nodes over a network**.



**1. Server Program (Node A)**

```
import  .io.*;
import  .net.*;
public class NodeAServer {
public static void main(String[] args) {
  try {
// Create a server socket on port 9000
   ServerSocket server = new ServerSocket(9000);
   System.out.println("Node A is waiting for a
connection...");
 // Wait for a client to connect
  Socket connection = server.accept();
 // Read message from the client
  BufferedReader in = new BufferedReader(
  new InputStreamReader(connection.getInputStream()));
  String message = in.readLine();
  System.out.println("Received: " + message);
  // Close connection
  connection.close();
  server.close();
  }
  catch (Exception e)
{
   e.printStackTrace();}}}
```

**How the Program Works**

**Step 1: Start Node A (Server)**

The server opens port **9000** and waits for a connection.

Node A is waiting for a connection...

**Step 2: Start Node B (Client)**

The client connects to Node A and sends:

Hello from Node B!

**Step 3: Server Receives the Message**

Node A receives the message and displays it.

Received: Hello from Node B!

**2. Client Program (Node B)**

```
import  .io.*;
import  .net.*;
public class NodeBClient {
public static void main(String[] args) {
try {
// Connect to Node A
   Socket socket = new Socket("localhost", 9000);
  // Send message to the server
   PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
   out.println("Hello from Node B!");
   System.out.println("Message sent to Node A.");
   // Close connection
   socket.close();
   }
catch (Exception e)
{
  e.printStackTrace();
 }
  }
 }
```

**Output on Node A (Server)**

1 -Node A is waiting for a connection...
2 -Received: Hello from Node B!

**Output on Node B (Client)**
Message sent to Node A.

# LECTURE 2 - PART A | PROGRAM #2 — ARCHITECTURAL FLOW

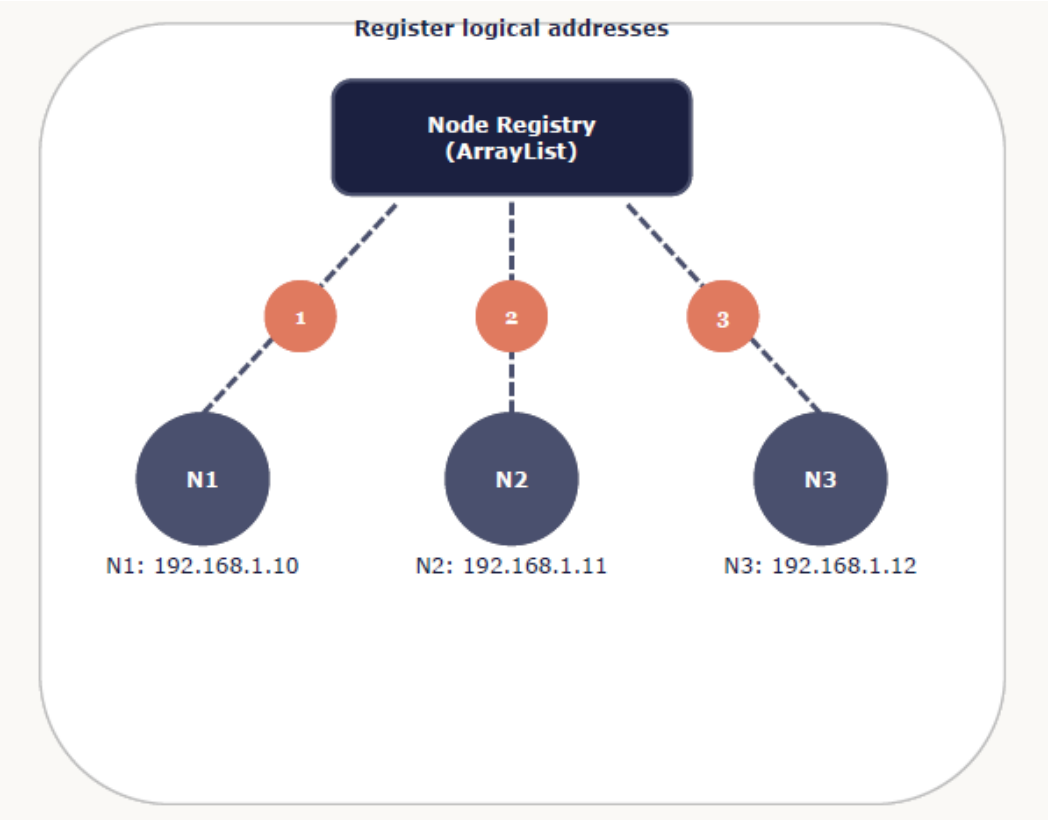
**Naming Service directory matches logical names with nodes**

## SIMPLE DESCRIPTION

a simple program that simulates a **Naming Service (Node Registry)** in a distributed system. The program creates several nodes, stores them in an ArrayList, and displays information about all registered nodes.

## FUNCTION OF THE PROGRAM

This program demonstrates a **simple naming service or node registry** in a distributed system. It maintains information about available nodes (servers/computers) and allows their details to be displayed. In real distributed systems, a naming service helps clients locate services and resources without needing to know their exact physical locations.



```
import  .util.ArrayList;
 class Node {
    String id;
    String address;
// Constructor
    Node(String id, String address) {
    this.id = id;
    this.address = address;
  }
   // Display node information
 @Override
 public String toString() {
      return "Node ID: " + id + ", Address: " + address;
 }
  }
```

**How the Program Works**

A Node class is created to represent a node in the distributed system.

Each node has:

- A unique **ID** (e.g., N1).
- An **IP address** (e.g.,  2.168.1.10).

An ArrayList named registry stores all nodes.

Three nodes are added to the registry.

A for-each loop iterates through the list and prints the details of each node.

```
public class NodeRegistry {
public static void main(String[] args) {
 // Create a list to store nodes
      ArrayList<Node> registry = new ArrayList<>();
  // Add nodes to the registry
     registry.add(new Node("N1", " 2.168.1.10"));
     registry.add(new Node("N2", " 2.168.1.11"));
     registry.add(new Node("N3", " 2.168.1.12"));
  // Display all registered nodes
     System.out.println("Registered Nodes:");

     for (Node node : registry) {
         System.out.println(node);
      }
  }
}
 `
```

**Sample Output**

1 Registered Nodes:
2 Node ID: N1, Address:  2.168.1.10
3 Node ID: N2, Address:  2.168.1. 11
4 Node ID: N3, Address:  2.168.1.12

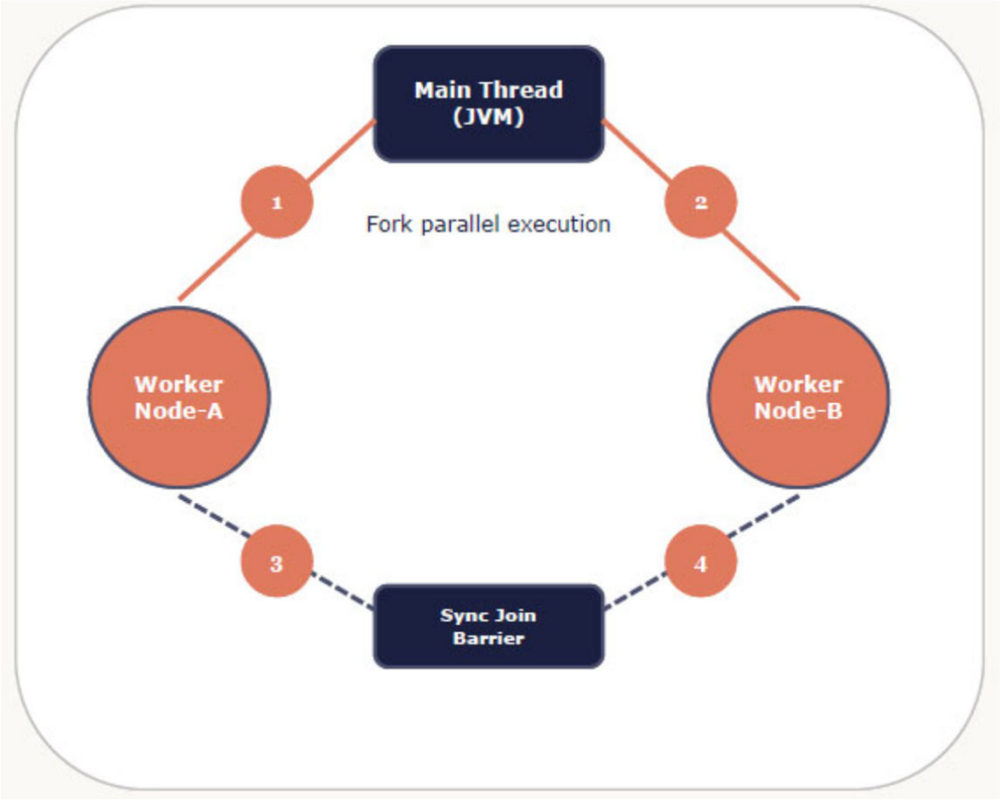
# LECTURE 2 - PART A | PROGRAM #3 — ARCHITECTURAL FLOW

**Threads perform tasks concurrently without blocking**

## SIMPLE DESCRIPTION

**Simple Program: Concurrent Execution in a Distributed System**

This program demonstrates the **Concurrent Execution** characteristic of a distributed system. Two independent nodes, **Node-A** and **Node-B**, perform their tasks simultaneously using separate threads. The main program waits for both threads to complete before displaying a final message.




## FUNCTION OF THE PROGRAM

The program simulates two independent nodes in a distributed system executing tasks **concurrently**. By using separate threads, Node-A and Node-B can perform their work at the same time rather than sequentially. The join() method ensures that the main program waits until both nodes finish before printing:
All nodes have completed their work.
This demonstrates the concept of **parallel/concurrent execution**, a key characteristic of distributed systems.

```
public class ConcurrentExecutionDemo {
public static void main(String[] args) {
// Node-A task
     Thread nodeA = new Thread(() -> {
       for (int i = 1; i <= 3; i++) {
       System.out.println("Node-A processing task " + i);
     }
     });
// Node-B task
    Thread nodeB = new Thread(() -> {
      for (int i = 1; i <= 3; i++) {
          System.out.println("Node-B processing task " + i);
    }
    });
   // Start both nodes concurrently
       nodeA.start();
       nodeB.start();
  // Wait for both nodes to finish
  try {
       nodeA.join();
       nodeB.join();
  }
```

## STATEMENT-BY-STATEMENT EXPLANATION

**Class Declaration**

public class ConcurrentExecutionDemo {

Defines a class named ConcurrentExecutionDemo.

The program starts execution from this class.

**Main Method**

public static void main(String[] args) {

The entry point of the program.

Execution begins here.

**Create Thread for Node-A**

Thread nodeA = new Thread(() -> {

Creates a new thread representing **Node-A**.

The code inside the braces is the task executed by Node-A.

**Node-A Tasks**

1 for (int i = 1; i <= 3; i++) {

2 System.out.println("Node-A processing task " + i);

3 }

Simulates Node-A performing three tasks.

Prints a message for each task.

```
catch (InterruptedException e) {
    e.printStackTrace();
 }
   // Final message
    System.out.println("All nodes have completed their
work.");
 }
 }
```

## STATEMENT-BY-STATEMENT EXPLANATION

**Create Thread for Node-B**

1 Thread nodeB = new Thread(() -> {

Creates a second thread representing **Node-B**.

This thread runs independently of Node-A.

**Node-B Tasks**

1 for (int i = 1; i <= 3; i++) {

2 System.out.println("Node-B processing task " + i);

3 }

Simulates Node-B performing three tasks.

Prints a message for each task.

**Start Node-A**

nodeA.start();

Starts execution of Node-A's thread.

**Start Node-B**

nodeB.start();

Starts execution of Node-B's thread.

Both threads now run concurrently.

```
catch (InterruptedException e) {
    e.printStackTrace();
 }
   // Final message
    System.out.println("All nodes have completed their
work.");
 }
 }
```

## STATEMENT-BY-STATEMENT EXPLANATION

**Wait for Node-A**

nodeA.join();

Causes the main thread to wait until Node-A finishes.

**Wait for Node-B**

nodeB.join();

Causes the main thread to wait until Node-B finishes.

**Exception Handling**

1 catch (InterruptedException e) {
2 e.printStackTrace();
3 }

Handles the case where a waiting thread is interrupted.

**Final Completion Message**

System.out.println("All nodes have completed their work.");

Executes only after both nodes finish their tasks.

```
catch (InterruptedException e) {
    e.printStackTrace();
  }
   // Final message
    System.out.println("All nodes have completed their
work.");
  }
  }
```

**Sample Output**

Because threads execute concurrently, the order may vary each time the program runs.

One possible output is:

1 Node-A processing task 1
2 Node-B processing task 1
3 Node-A processing task 2
4 Node-B processing task 2
5 Node-A processing task 3
6 Node-B processing task 3
7 All nodes have completed their work.

Another possible output is:

1 Node-B processing task 1
2 Node-B processing task 2
3 Node-A processing task 1
4 Node-A processing task 2
5 Node-B processing task 3
6 Node-A processing task 3
7 All nodes have completed their work.

# LECTURE 2 - PART A | PROGRAM #4 — ARCHITECTURAL FLOW

**Echo Server receives messages and returns them back**

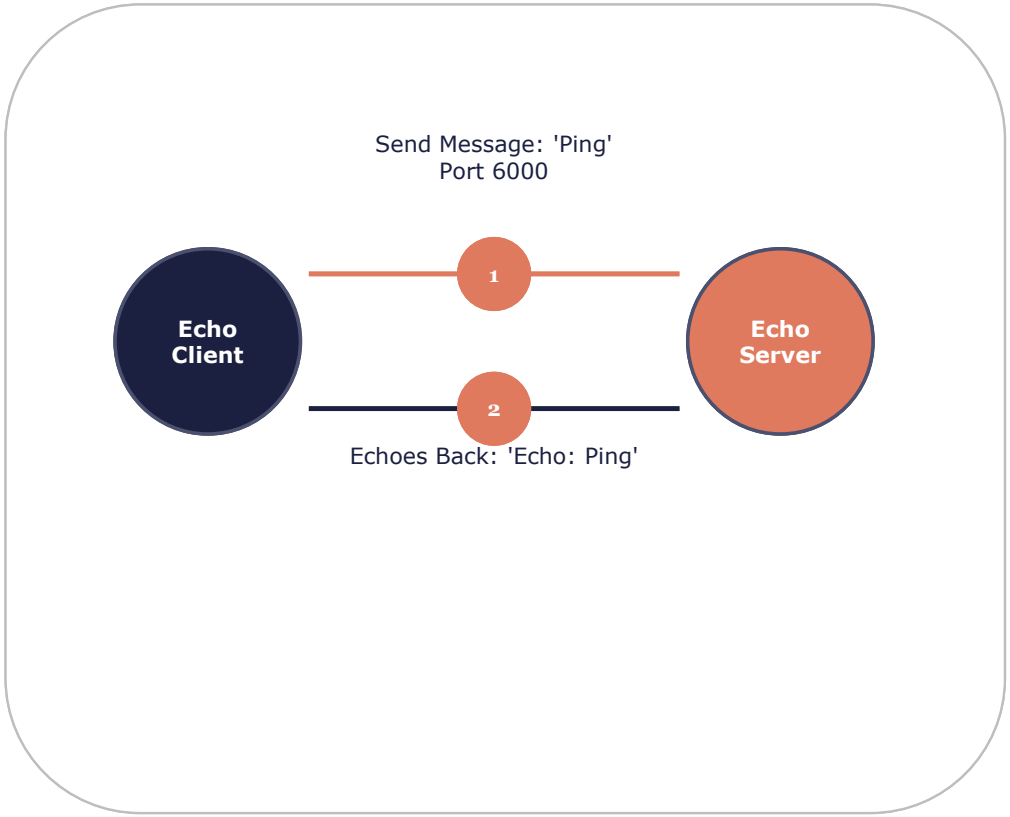
## SIMPLE DESCRIPTION

Here is a simple program that demonstrates a **Client-Server Architecture** and **Message Passing using TCP Sockets**. The server acts as an **Echo Server**, which receives a message from the client and sends it back with the prefix "Echo: ".

**Server Program (EchoServer. )**

## FUNCTION OF THE PROGRAM

This simple example illustrates how a client and server communicate through TCP sockets, with the server echoing back the client's message.



```
import  .io.*;
import  .net.*;
public class EchoServer {
public static void main(String[] args) {
try {
    // Create server socket on port 6000
     ServerSocket server = new ServerSocket(6000);
     System.out.println("Server is waiting for a client...");
    // Accept client connection
     Socket client = server.accept();
    // Input stream to receive data
     BufferedReader in = new BufferedReader(
      new InputStreamReader(client.getInputStream()));
    // Output stream to send data
      PrintWriter out = new
PrintWriter(client.getOutputStream(), true);
   // Read message from client
      String message = in.readLine();
    // Send echo response
      out.println("Echo: " + message);
    // Close connections
      client.close();
      server.close();
   }
 catch (Exception e) {
```

**How the Program Works**

The server starts and listens on **port 6000**.
The client connects to the server.
The client sends the message:
Ping
The server receives the message and creates a response:
Echo: Ping
The server sends the response back to the client.
The client receives and displays the response.

**Client Program (EchoClient. )**

```
import  .io.*;
import  .net.*;
public class EchoClient {
public static void main(String[] args) {
try {
// Connect to the server
     Socket socket = new Socket("localhost", 6000);

// Output stream to send data
     PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
// Input stream to receive response
    BufferedReader in = new BufferedReader(
 new InputStreamReader(socket.getInputStream()));
  out.println("Ping");
```

**Output on the Server Screen**

Server is waiting for a client...

**Output on the Client Screen**

Echo: Ping

**Final Result**

Echo: Ping

# Questions?

Thank you — see you in Lecture 3: Goals of Distributed Systems.